ATIVIDADE PRÁTICA – AULA 04

Operações *CRUD* no MongoDB

Banco de Dados Não Relacional · Prof.ª Lucineide · FATEC

- 9 Exercícios Práticos
- 1 Questão Discursiva
- Banco: dbcursos
- Coleção: alunos

CONTEXTO

Banco **dbcursos** criado, coleção **alunos** populada com os documentos da aula. A seguir, o script de inserção inicial utilizado como base para todos os exercícios.

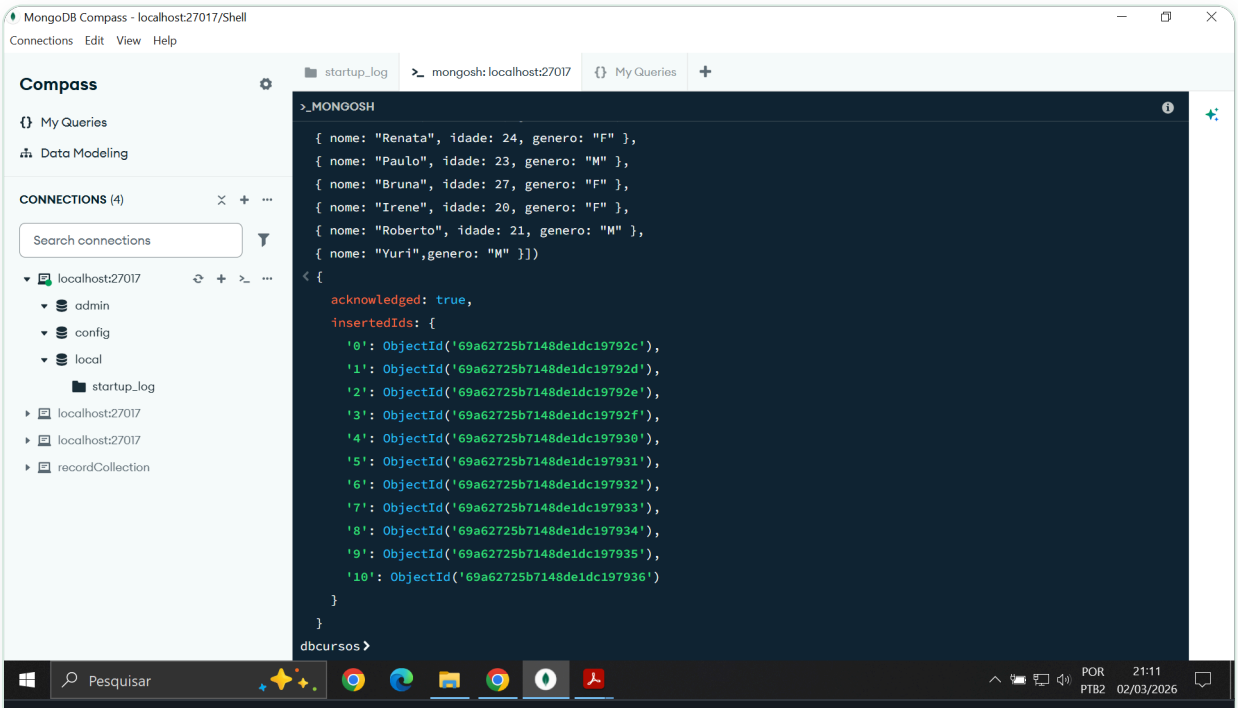
Inserção Inicial dos Dados — dbcursos.alunos

insertMany

MongoDB Shell — Populando a coleção alunos

```
use dbcursos

db.alunos.insertMany([
  { nome: "Pedro",   idade: 25, genero: "M" },
  { nome: "Ana",     idade: 20, genero: "F" },
  { nome: "Maria",   idade: 21, genero: "F" },
  { nome: "Lucas",   idade: 28, genero: "M" },
  { nome: "João",    idade: 22, genero: "M" },
  { nome: "Renata",  idade: 24, genero: "F" },
  { nome: "Paulo",   idade: 23, genero: "M" },
  { nome: "Bruna",   idade: 27, genero: "F" },
  { nome: "Irene",   idade: 20, genero: "F" },
  { nome: "Roberto", idade: 21, genero: "M" },
  { nome: "Yuri",    genero: "M" }
])
```



Resultado da inserção dos 11 documentos na coleção alunos

EXERCÍCIO 1 Cadastro e Listagem de Alunos

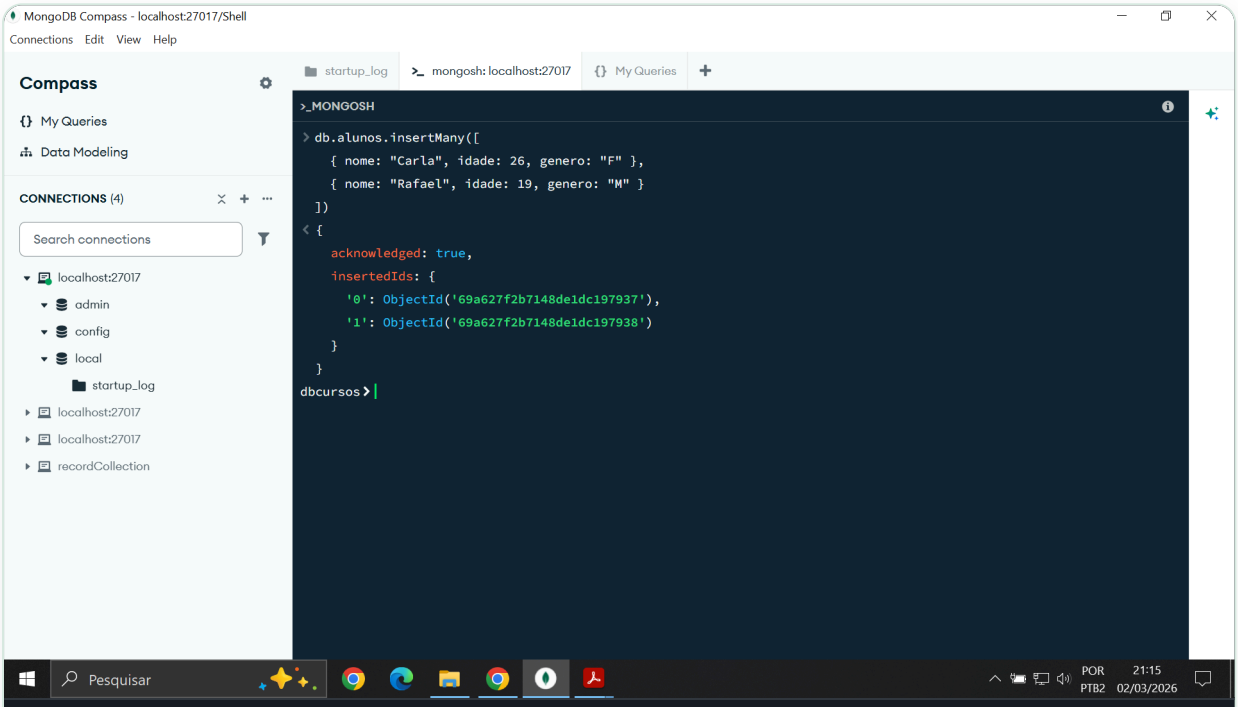
1 Inserir dois novos alunos e listar toda a coleção

insertMany · find

Cadastro de dois novos alunos — um do gênero feminino (idade 26) e outro do gênero masculino (idade 19) — seguido da listagem completa da coleção.

MongoDB Shell — Exercício 1 · Inserção

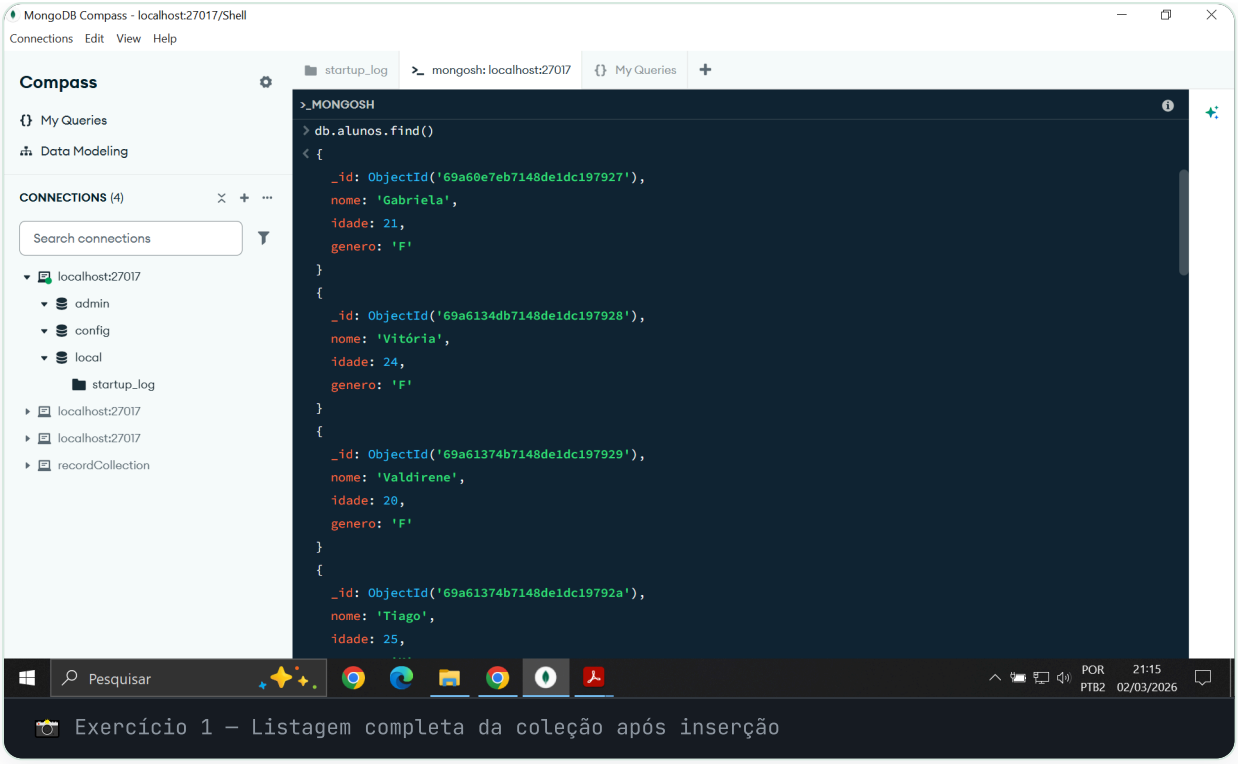
```
// Cadastrar dois novos alunos
db.alunos.insertMany([
  { nome: "Carla",  idade: 26, genero: "F" },
  { nome: "Rafael", idade: 19, genero: "M" }
])
```



Exercício 1 — Inserção: acknowledged: true, dois novos ObjectIds gerados

MongoDB Shell — Exercício 1 · Listagem completa

```
// Listar todos os documentos da coleção
db.alunos.find()
```



Exercício 1 — Listagem completa da coleção após inserção

EXERCÍCIO 2 Consulta com Filtro e Projeção

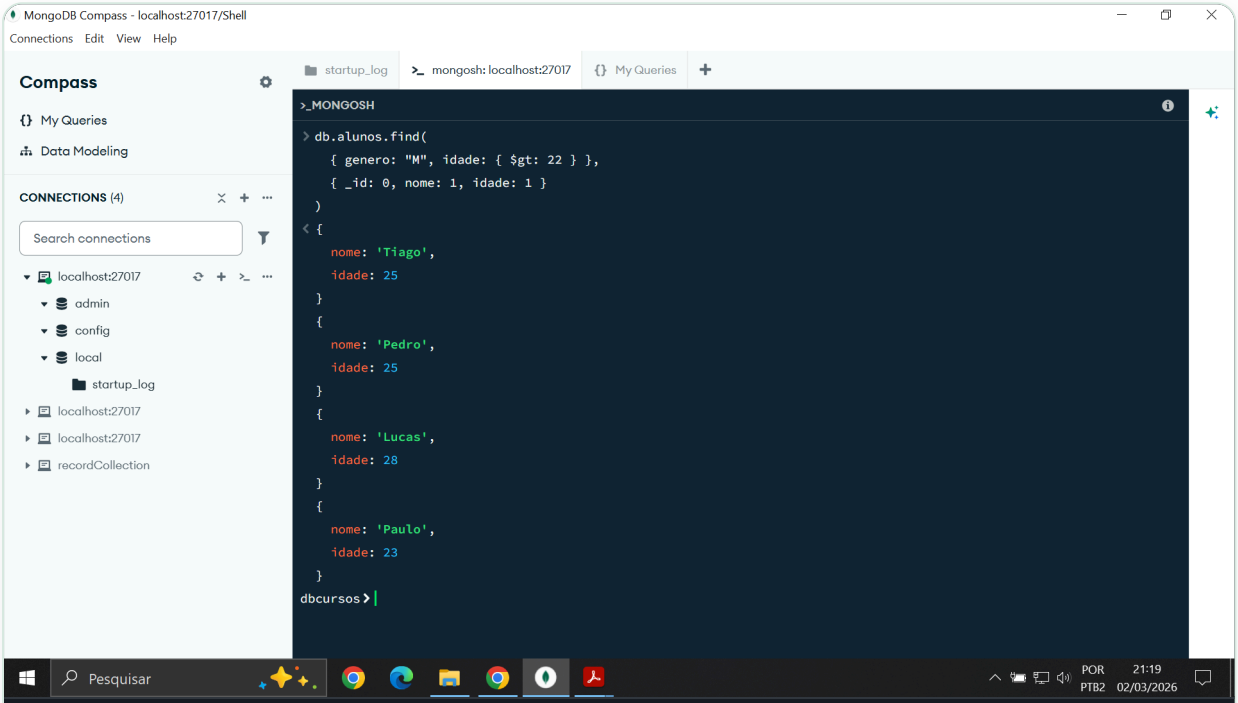
2 Alunos do gênero masculino com idade maior que 22

find · $gt · projection

Filtro combinando gênero masculino **E** idade maior que 22, exibindo apenas **nome** e **idade**, ocultando o campo **_id**.

MongoDB Shell — Exercício 2

```
db.alunos.find(
  { genero: "M", idade: { $gt: 22 } },
  { _id: 0, nome: 1, idade: 1 }
)
```



Exercício 2 — Alunos masculinos com idade > 22: Tiago, Pedro, Lucas, Paulo

EXERCÍCIO 3 Intervalo de Idades com Ordenação

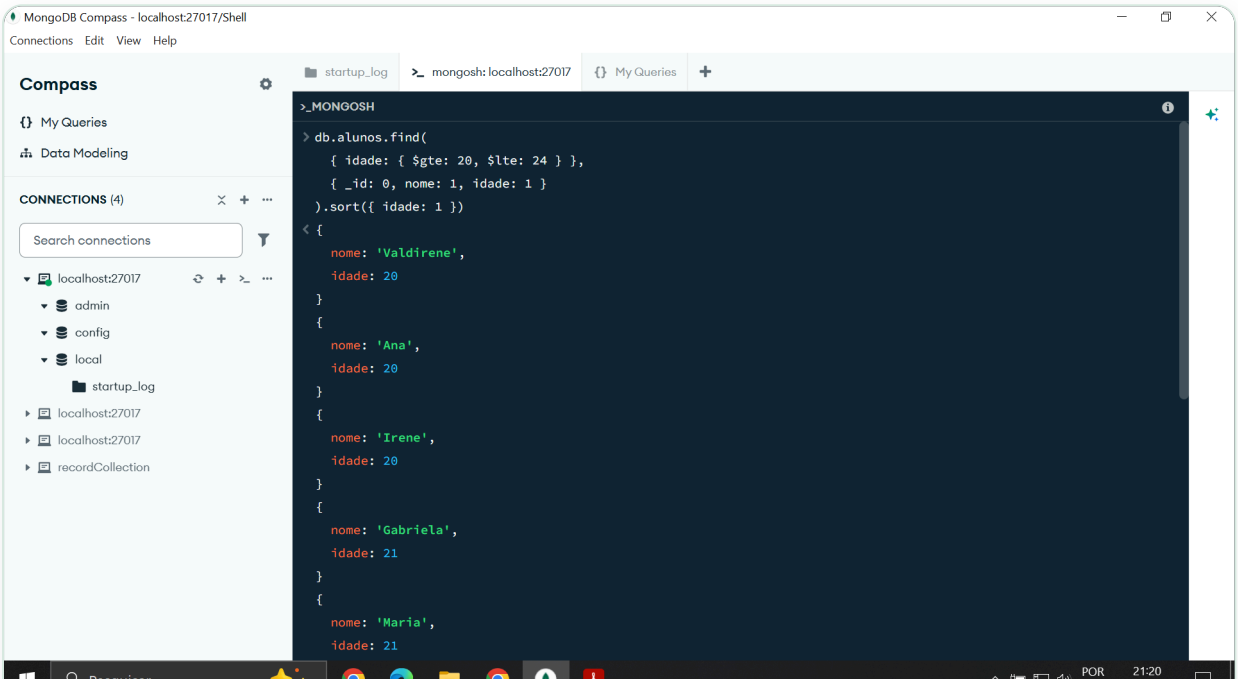
3 Alunos entre 20 e 24 anos, ordenados por idade crescente

find · $gte · $lte · sort

Consulta com operadores de comparação **$gte** (maior ou igual) e **$lte** (menor ou igual), com resultado ordenado por idade de forma crescente, mostrando apenas **nome** e **idade**.

MongoDB Shell — Exercício 3

```
db.alunos.find(
  { idade: { $gte: 20, $lte: 24 } },
  { _id: 0, nome: 1, idade: 1 }
).sort({ idade: 1 })
```



Exercício 3 — Alunos de 20-24 anos ordenados por idade crescente

EXERCÍCIO 4 Busca por Lista de Nomes

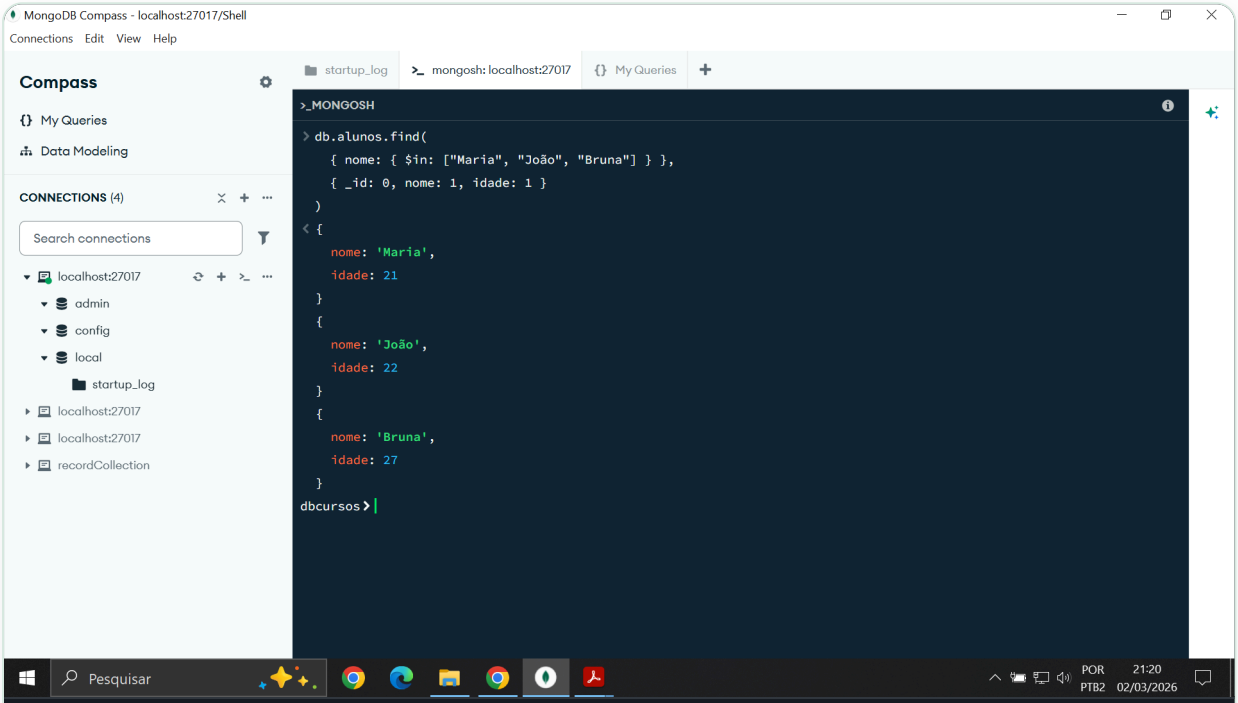
4 Listar alunos: Maria, João e Bruna

find · $in

Utilização do operador **$in** para buscar múltiplos valores em um único campo, exibindo **nome** e **idade**.

MongoDB Shell — Exercício 4

```
db.alunos.find(
  { nome: { $in: ["Maria", "João", "Bruna"] } },
  { _id: 0, nome: 1, idade: 1 }
)
```



Exercício 4 — Resultado: Maria (21), João (22) e Bruna (27)

EXERCÍCIO 5 Busca com Expressão Regular

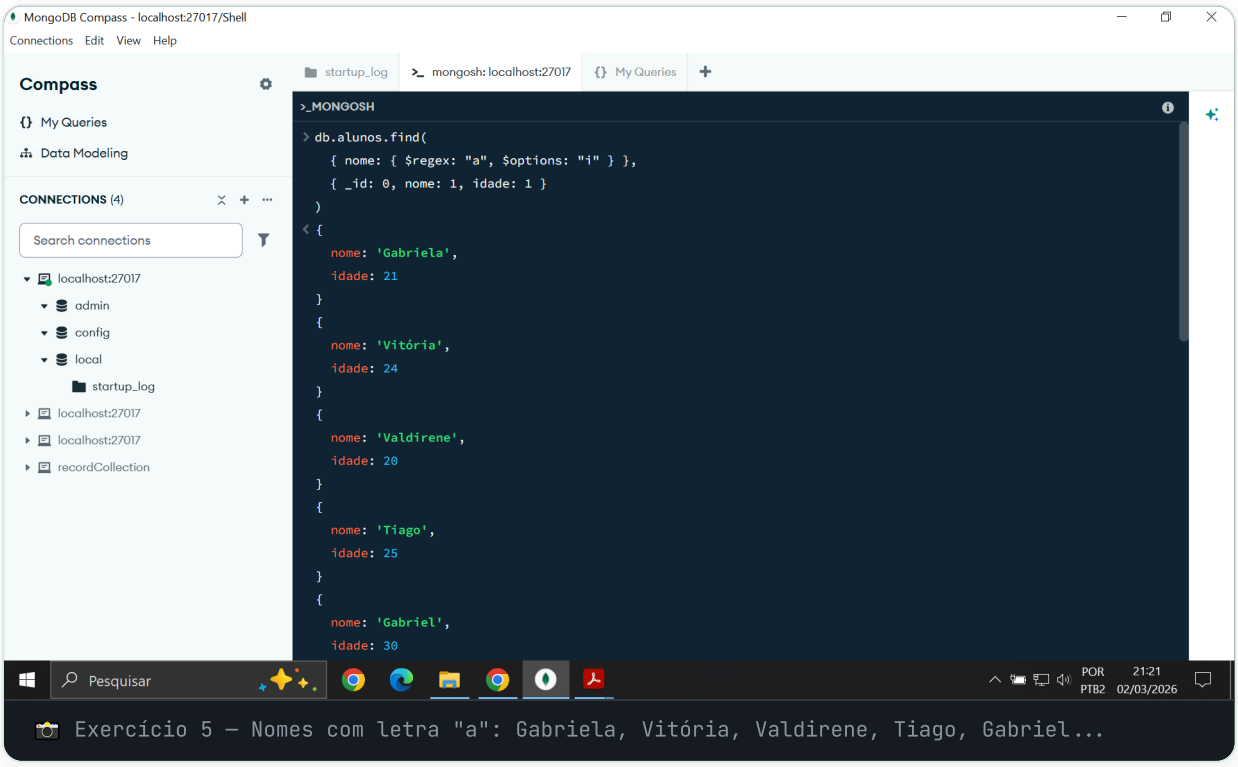
5 Alunos cujo nome contenha a letra "a" (case-insensitive)

find · $regex · $options

Uso do operador **$regex** com a opção **"i"** para ignorar diferença entre maiúsculas e minúsculas, encontrando todos os nomes que contenham a letra "a" em qualquer posição.

MongoDB Shell — Exercício 5

```
db.alunos.find(
  { nome: { $regex: "a", $options: "i" } },
  { _id: 0, nome: 1, idade: 1 }
)
```



Exercício 5 — Nomes com letra "a": Gabriela, Vitória, Valdirene, Tiago, Gabriel...

EXERCÍCIO 6 Atualização de Documento

6 Atualizar a idade da aluna Ana para 30 anos

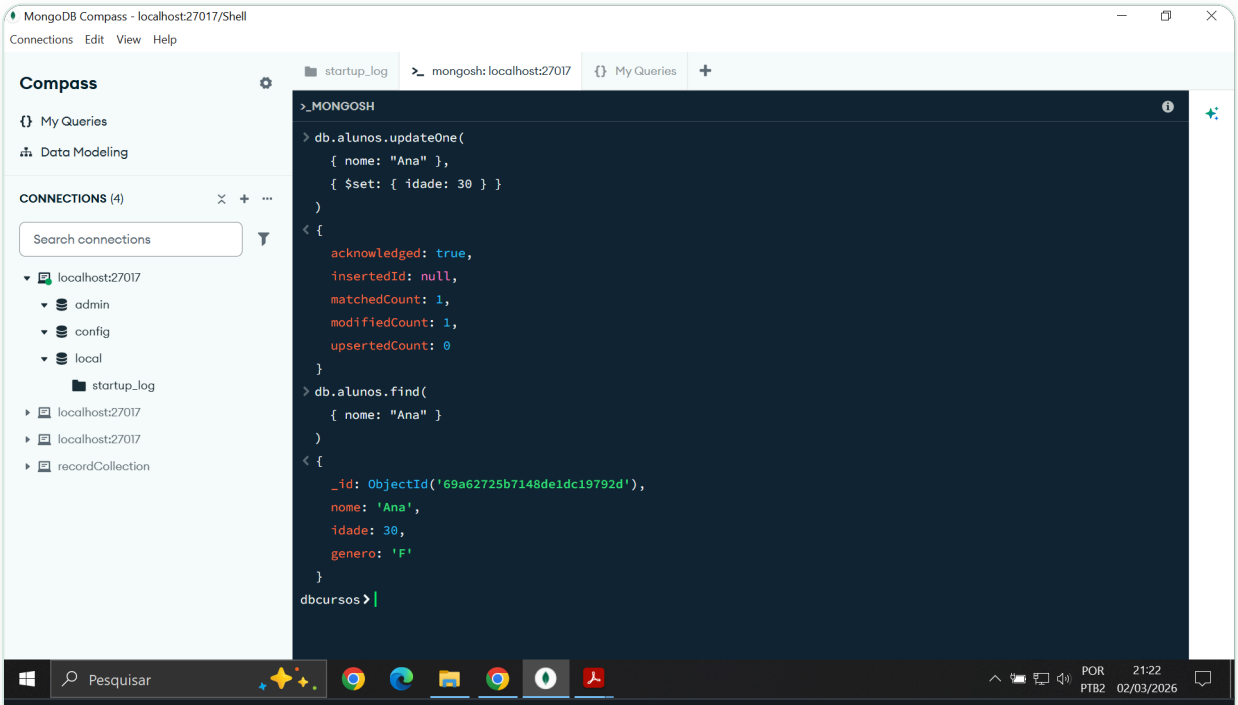
updateOne · $set

Uso do **updateOne** com o operador **$set** para alterar apenas o campo **idade** da aluna Ana, preservando todos os outros campos. Em seguida, listagem do documento atualizado para confirmação.

MongoDB Shell — Exercício 6

```
// Atualizar idade de Ana para 30
db.alunos.updateOne(
  { nome: "Ana" },
  { $set: { idade: 30 } }
)

// Verificar o documento atualizado
db.alunos.find(
  { nome: "Ana" }
)
```



Exercício 6 — matchedCount: 1, modifiedCount: 1 · Ana agora com idade: 30

EXERCÍCIO 7 Atualização em Massa

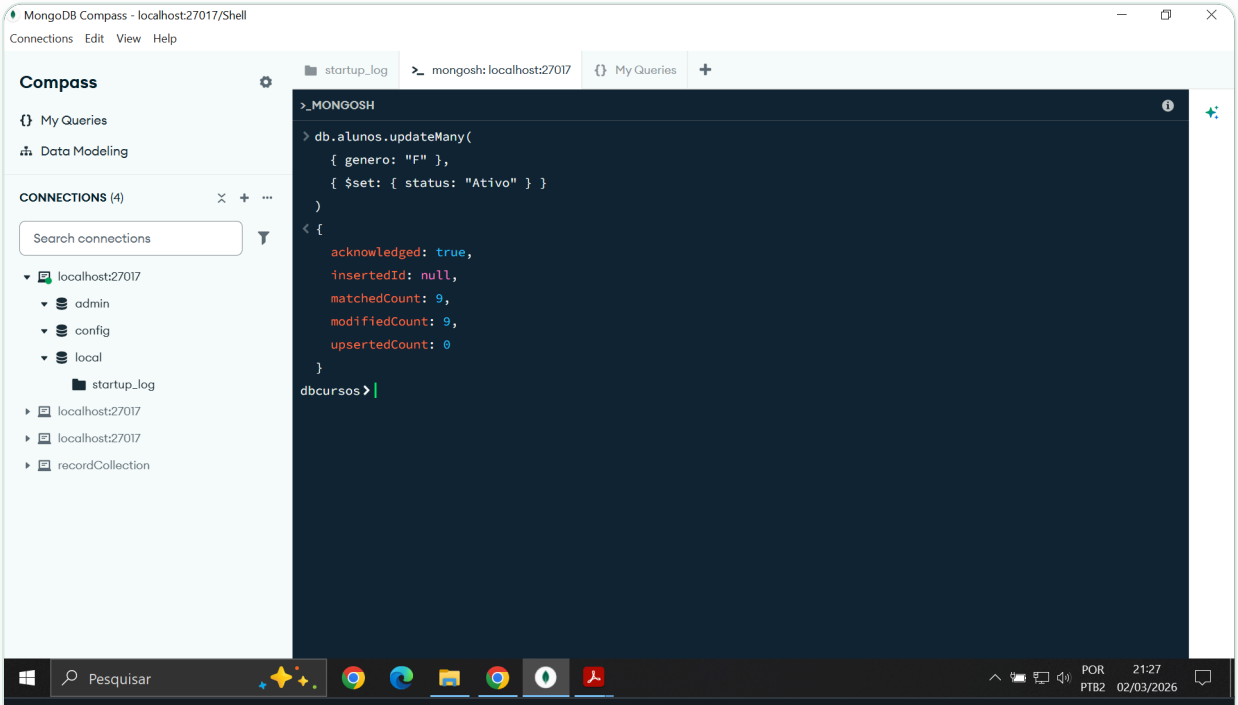
7 Adicionar campo status: "Ativo" para todas as alunas

updateMany · $set

Uso do **updateMany** para aplicar a mesma alteração em múltiplos documentos simultaneamente — adicionando o campo **status: "Ativo"** para todas as alunas do gênero feminino. Em seguida, listagem confirmando a atualização.

MongoDB Shell — Exercício 7 · Atualização em massa

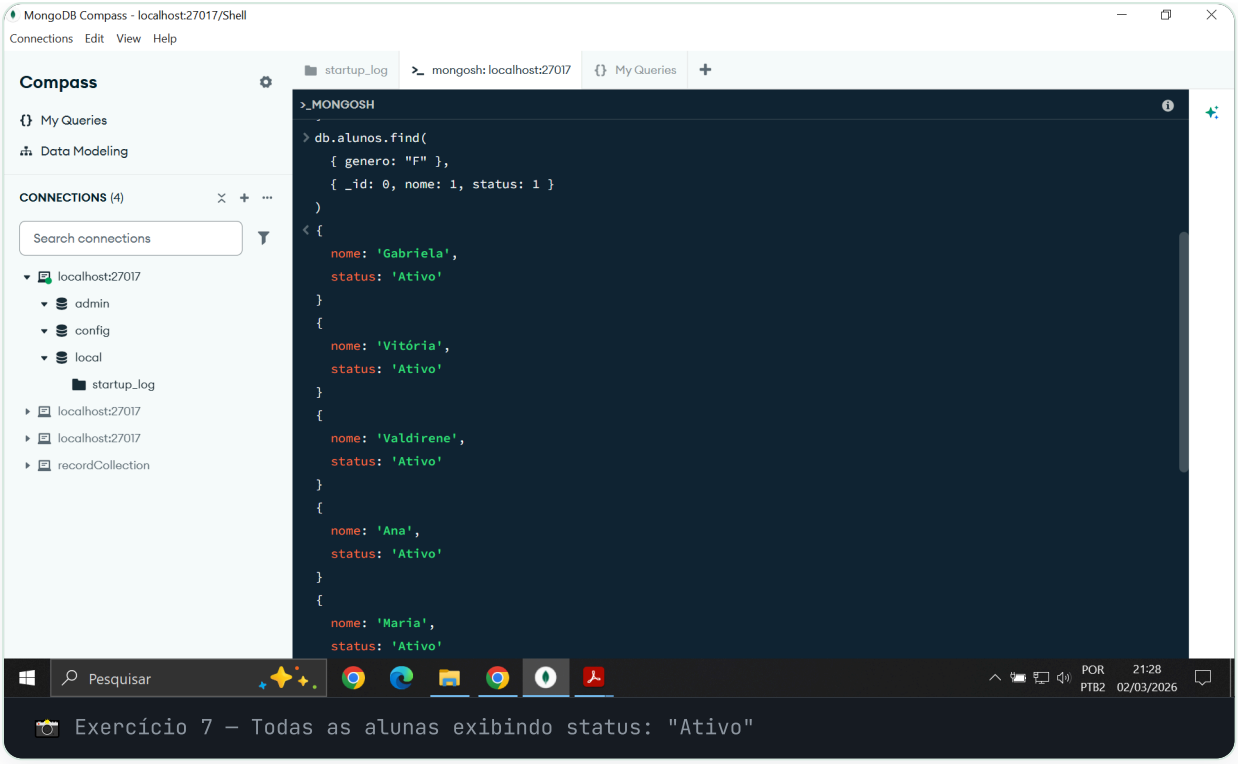
```
// Adicionar status "Ativo" para todas as alunas (gênero F)
db.alunos.updateMany(
  { genero: "F" },
  { $set: { status: "Ativo" } }
)
```



Exercício 7 — matchedCount: 9, modifiedCount: 9 · Todas as alunas atualizadas

MongoDB Shell — Exercício 7 · Listagem de confirmação

```
// Listar alunas mostrando apenas nome e status
db.alunos.find(
  { genero: "F" },
  { _id: 0, nome: 1, status: 1 }
)
```



EXERCÍCIO 8 Remoção de Documento

8 Remover o aluno "Yuri" e listar a coleção

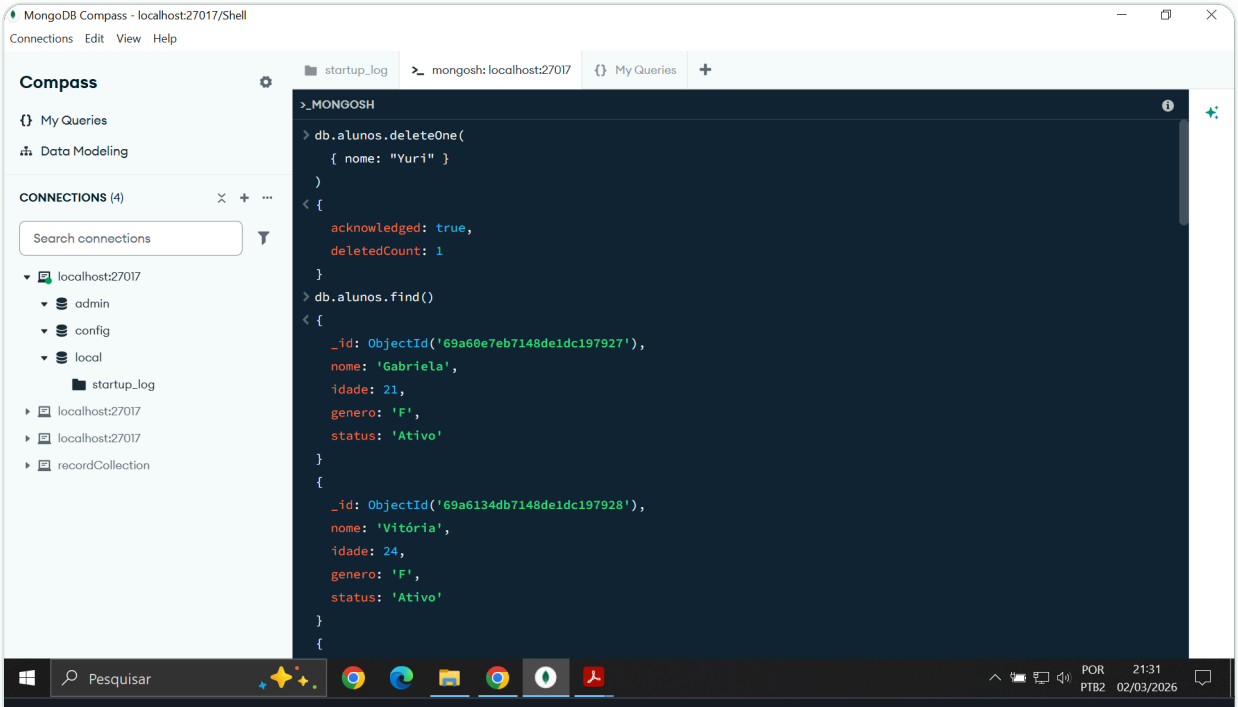
deleteOne · find

Utilização do **deleteOne** para remover o documento do aluno Yuri, seguida da listagem completa da coleção para comprovar a exclusão.

MongoDB Shell — Exercício 8

```
// Remover o aluno Yuri
db.alunos.deleteOne(
  { nome: "Yuri" }
)

// Listar todos os alunos para confirmar exclusão
db.alunos.find()
```



Exercício 8 — deletedCount: 1 · Yuri removido, coleção listada sem o documento

EXERCÍCIO 9 Paginação com Sort, Skip e Limit

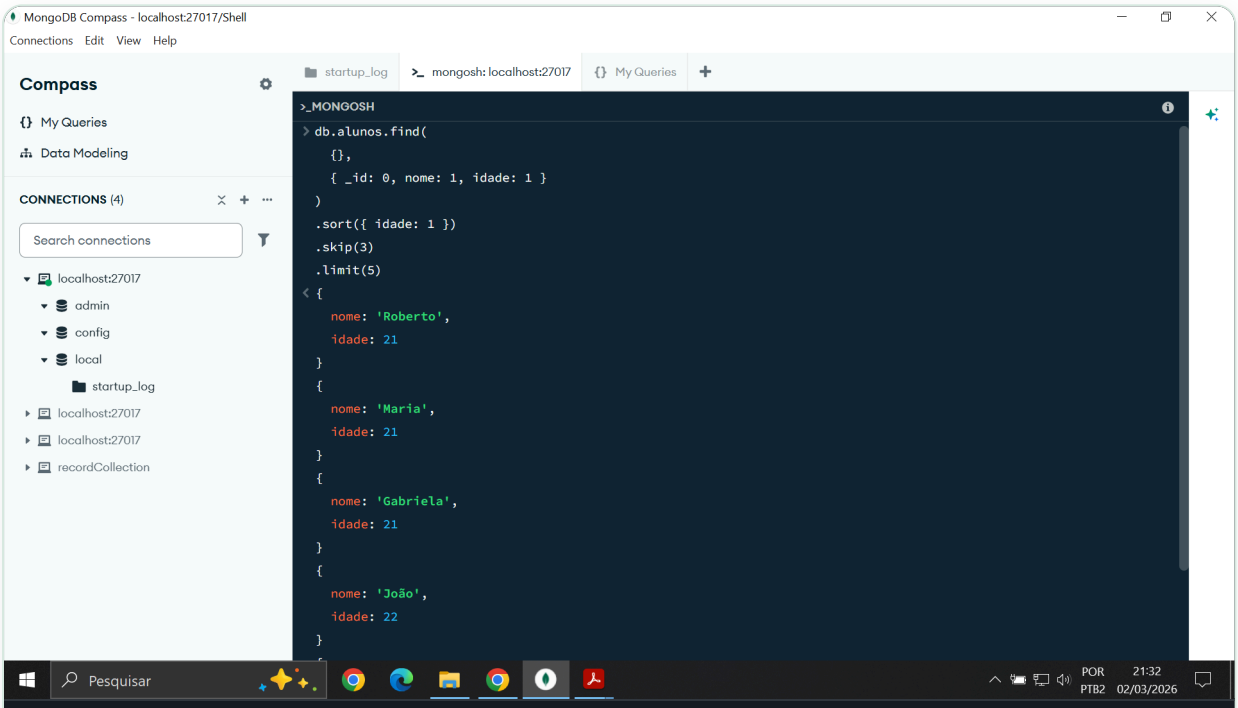
9 Ordenar por idade, pular 3 primeiros, retornar 5

sort · skip · limit

Combinação de **sort** (ordenação crescente por idade), **skip** (ignorar os 3 primeiros registros) e **limit** (retornar no máximo 5 documentos) — técnica fundamental para paginação de resultados.

MongoDB Shell — Exercício 9

```
db.alunos.find(
  {},
  { _id: 0, nome: 1, idade: 1 }
).sort({ idade: 1 })
 .skip(3)
 .limit(5)
```



Exercício 9 — 5 documentos retornados após pular os 3 primeiros da ordem crescente

EXERCÍCIO 10 Questões Discursivas

10 Respostas discursivas sobre operações CRUD

Discursiva

1 Qual a principal diferença entre `updateOne()` e `updateMany()` ?

O `updateOne()` localiza o **primeiro documento** que satisfaz o filtro e aplica a alteração somente nele, encerrando a operação em seguida — independente de quantos documentos correspondam ao critério. Já o `updateMany()` percorre **todos os documentos** que satisfazem o filtro e aplica a mesma modificação em cada um deles. A escolha entre os dois depende da intenção: quando se quer atualizar um registro específico (por exemplo, alterar o e-mail de um usuário pelo seu ID) usa-se `updateOne()`; quando se quer aplicar uma regra a um grupo de documentos (como marcar todos os pedidos como "entregue") usa-se `updateMany()`.

2 Qual a diferença entre `deleteOne()` e `deleteMany()` ?

O `deleteOne()` remove apenas o **primeiro documento** encontrado que corresponda ao filtro informado, retornando `deletedCount: 1` no máximo. O `deleteMany()`, por sua vez, remove **todos os documentos** que satisfaçam o critério do filtro de uma só vez, podendo retornar um `deletedCount` maior que 1. Um uso descuidado de `deleteMany()` com um filtro muito abrangente (como `{}` vazio) pode esvaziar a coleção inteira, por isso exige atenção especial antes de ser executado em produção.

3 Em qual situação você utilizaria `$regex` em um sistema real?

O `$regex` é utilizado quando se precisa realizar buscas textuais parciais ou flexíveis em campos string, sem exigir correspondência exata. Situações práticas incluem: campos de **busca/pesquisa** em interfaces de usuário (como um campo "Pesquisar aluno por nome"), onde o usuário digita parte do nome e o sistema deve retornar todos os registros que contenham aquele trecho; **validação de formato** de dados na consulta (ex.: filtrar e-mails que terminam em "@fatec.sp.gov.br"); e **migração ou auditoria de dados**, encontrando documentos que possuem um padrão específico em algum campo de texto. É importante notar que o `$regex` sem índice de texto pode ser lento em coleções muito grandes, então em sistemas de alta escala o ideal é combiná-lo com índices ou usar o operador `$text` para buscas mais complexas.